Nested loop:

A loop inside of another loop is called as nested loop

Common situations:

- Working with tables / matrices (rows & columns)
- Comparing elements (pair-wise comparison)
- Generating combinations
- Pattern printing
- Processing grouped or hierarchical data

-->W.a.p to print following series

```
1 2 3
1 2 3
```

no.of rows --> 2
no.of columns --> 3

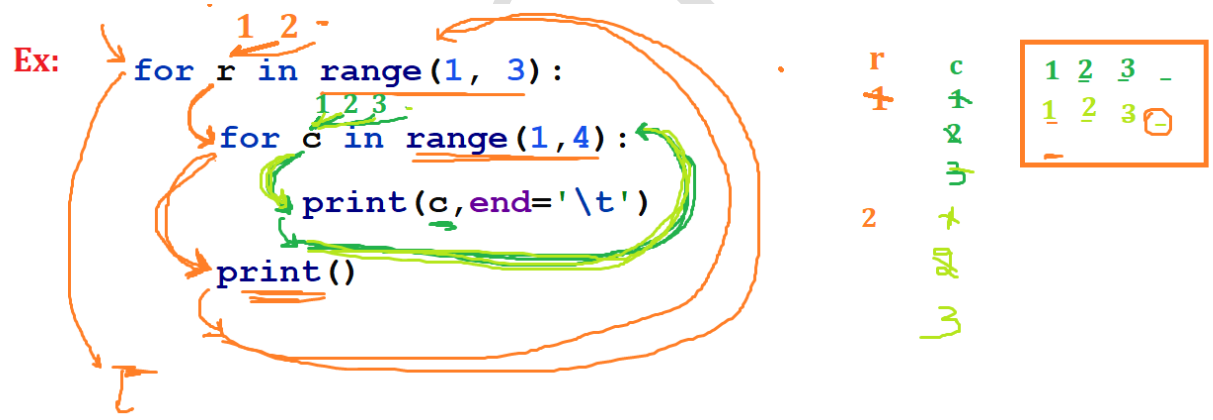
```
1 2 3 4 5
1 2 3 4 5
```

Syntax:

```
for variable in range(1, rowsize+1):
    for variable in range(1, colsize+1):
        Statement-1
        .....................
    print( )
```

Ex:

```
for r in range(1, 3):
    for c in range(1,4):
        print(c,end='\t')
    print()
```



-->W.a.p to print following series

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
r  c
1  1
2  2
3  3
4  4
5  5
```

```
for r in range(1,6):
    for c in range(1,r+1):
        print(c,end='\t')
    print()
```

--> W.a.p to check given number is prime number or not if it is prime number then only print that value otherwise don't print that value

```
n = int(input('Enter any Number : '))
count = 0
for i in range(1,n+1):
    if n%i==0:
        count +=1
if count==2:
    print(n)
```

--> W.a.p to print prime numbers between 1 to 100

```
for n in range(1, 101):
    count = 0
    for i in range(1,n+1):
        if n%i==0:
            count +=1
    if count==2:
        print(n)
```

while loop

--> If you don't know number of iterations, but you need to execute certain statements multiple times based on condition then use while loop

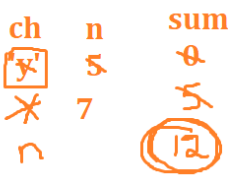
Syntax:

```
while condition:
    Statement-1
    .....
Statement-2
```

False

--> W.a.p to read multiple values from user then find sum

```
sum = 0
ch='y'
while ch=='y':
    n = int(input('Enter any Number : '))
    sum = sum + n
    ch = input('Do you want to continue(y/n): ')
print('Sum is : ',sum)
```



-->W.a.p to read multiple numbers from user then find sum of even numbers only

```
sum = 0
ch='y'
while ch=='y':
    n = int(input('Enter any Number : '))
    if n%2==0:
        sum = sum + n
    ch = input('Do you want to continue(y/n): ')
print('Sum is : ',sum)
```

Transfer Statements :

-->To Move control from one place to another place we need to use Transfer Statements

1. break :

--> break is a keyword which is used only in loops
--> If you want to stop iterations then use break
--> In loops, whenever break occured then control stops iterations then goes outside of loop

Ex:

```
for i in range(1,6):
    if i%2==0:
        break
    print(i)
print('Thank you')
```

1 2 3 4 5

i		
1	1%2==0	1
2	2%2==0 ✓	-

2. continue

--> continue is a keyword, which is used only in looping statements
--> Whenever continue occured then it stops current iteration then continue for next iteration

Ex:

```
for i in range(1, 6):
    if i%2 == 0:
        continue
    print(i)
print('Thank you')
```

1 2 3 4 5

i		
1	1%2==0	1
2	2%2==0	3
3	3%2==0	5
4	4%2==0	Thank you
5	5%2==0	

➔ Consider a=[5, 9, 12, 8, 10], now read a number if number available then print 'Your number found' otherwise print 'Sorry number not available'

```
a = [5, 9, 12, 8, 10]
n = int(input('Enter any Number : '))
c = 0
for i in a:
    if i==n:
        c += 1
        break
if c==0:
    print('Sorry Number is not Available')
else:
    print('Number is Available')
```

for-else

➔ If for loop is executed successfully without any break the only if you want to execute some statements then use for-else
➔ for loop runs normally
➔ else block runs only if the loop did NOT break
➔ If a break happens → else will NOT execute

Syntax:

```
for item in iterable:
    if condition:
        break
else:
    # runs only if loop completes fully (no break)
```

Ex:

```
a = [5, 9, 12, 8, 10]
n = int(input('Enter any Number : '))
for i in a:
    if i==n:
        print(f'{n} Available')
        break
else:
    print(f'{n} not Available')
```

Checking if a Data File Contains Errors or not

Imagine you're scanning records for invalid data. If any -ve value comes then it is invalid data

```
data = [10, 20, 30, -5, 40]
for value in data:
    if value < 0:
        print("Invalid data found:", value)
        break
else:
    print("All data is valid")
```

Checking Duplicate Data in Dataset

```
data = [1, 2, 3, 4, 5]
new_value = 3

for val in data:
    if val == new_value:
        print("Duplicate found")
        break
else:
    print("No duplicate, safe to insert")
```

Pipeline Step Validation (ETL Scenario)

```
steps_status = ["success", "success", "failed", "success"]
for step in steps_status:
    if step == "failed":
        print("Pipeline failed")
        break
else:
    print("Pipeline executed successfully")
```

Use for-else when:

- ➔ You are searching for something
- ➔ You want to confirm absence of a condition
- ➔ You need validation checks across datasets
- ➔ You want clean logic without extra flags